



HARAMAYA UNIVERSITY

College of Computing and Informatics
Department of Software Engineering

Student Academic Record Management System

SOFTWARE ENGINEERING PROJECT REPORT

COURSE

Advanced Database

INSTRUCTOR

Mr Gizacho B.

SUBMISSION DATE

April 2026

GROUP MEMBERS

Chala Gobena	0964/16
Amenti Liben	0686/16
Boka Jirenga	0931/16
Damoze Motuma	0959/16
Jalane Feyisa	1663/16
Dawit Marsha	1026/16

Abstract

This project presents a **Student Academic Record Management System** whose main contribution is the design and implementation of a well-structured academic database. The core problem addressed is the weakness of manual result processing in educational institutions, where marks, totals, ranks, and report sheets are often managed using paper files or disconnected spreadsheets. Such methods are difficult to validate, vulnerable to inconsistency, and unsuitable for secure multi-user access. To solve this problem, the project models academic data using a normalized relational schema implemented in **Supabase PostgreSQL**.

The database stores the fundamental academic entities of the institution, including students, teachers, subjects, classes, marks, and user profiles. The design applies **primary keys, foreign keys, composite keys, uniqueness constraints, check constraints, SQL views, stored functions, triggers, and Row-Level Security (RLS)**. Report values such as total, average, rank, and pass or fail status are derived dynamically from stored marks instead of being duplicated in base tables. This approach improves consistency, reduces update anomalies, and ensures that reports remain synchronized with authoritative source data.

Although a lightweight web interface exists to submit and display data, the academic value of the project lies primarily in the database layer. The web layer acts only as a thin access mechanism; the essential business rules, data integrity rules, reporting logic, and security controls are enforced in the database. For that reason, this report focuses mainly on database understanding, schema design, SQL implementation, normalization, performance, and secure data access.

Table of Contents

1. Introduction
 2. Requirement Analysis
 3. Database-Oriented System Design
 4. Entity Relationship Diagram
 5. Database Design and Normalization
 6. SQL Implementation and Database Operations
 7. Security, RLS, and Exception Handling
 8. Testing and Validation
 9. Database-Relevant Screenshots and Evidence
 10. Unique and Innovative Database Features
 11. Conclusion
 12. Appendix
-

1. Introduction

Student result management is one of the most important database problems in academic administration. At the end of every semester, institutions must record marks, maintain correct relationships between students and subjects, calculate totals and averages, assign ranks, determine pass or fail status, and produce readable reports. If these operations are handled manually, errors occur easily and data integrity becomes difficult to maintain.

The **Student Academic Record Management System** was developed to address this problem through a relational database approach. Instead of storing repeated academic information in multiple files, the system stores each data item once in a structured schema and derives report values from the stored transactions. The project therefore demonstrates how database design principles can be applied to a real institutional problem.

1.1 Problem Statement

Educational institutions need a secure and consistent way to store academic records and generate result reports without relying on manual calculations or unstructured spreadsheets. A database-centered solution is required to maintain correct relationships between students, teachers, classes, subjects, and marks while enforcing integrity and security.

1.2 General Objective

To design and implement a normalized relational database that supports secure storage, processing, and reporting of student academic records.

1.3 Specific Objectives

- Analyze the student result management problem from a database perspective.
- Design an ER model and relational schema with appropriate keys and cardinalities.
- Apply normalization up to Third Normal Form.
- Implement integrity constraints, referential actions, SQL views, stored functions, and triggers.
- Enforce secure access through Row-Level Security and controlled database functions.
- Generate academic reports dynamically using SQL aggregation and ranking logic.
- Use a minimal interface only as a mechanism to access database operations safely.

1.4 Scope of the Project

The project covers:

- student master data,
- teacher master data,
- class and subject definition,
- mark storage,
- result summary generation,
- role-based record visibility,
- password change support,
- database-backed report generation.

The current project does not cover:

- attendance management,
- finance management,
- timetable management,
- detailed historical transcript archiving across many terms,
- institutional messaging modules.

2. Requirement Analysis

The requirements of the system were identified from the data and control needs of academic result management. The focus of this analysis is not interface design, but the database objects, data relationships, and business rules necessary to support correct academic processing.

2.1 Functional Requirements

Database Function Area	Required Capability	Implemented Design
Student Data Management	Store learner identity and academic placement	Student records are stored in <code>students</code> with class, grade, academic year, and semester attributes
Teacher Data Management	Store teacher identity and teaching scope	Teacher records are stored in <code>teachers</code> , linked to subjects and optionally as homeroom teacher
Subject Management	Store curriculum subjects	Subjects are stored in <code>subjects</code> with unique names and total marks
Class Management	Organize class groups	Classes are stored in <code>classes</code> with optional homeroom teacher reference

Database Function Area	Required Capability	Implemented Design
Mark Transaction Management	Store each subject mark once	<code>marks</code> stores one mark per (<code>student_id</code> , <code>subject_id</code>) pair
Reporting	Compute totals, averages, rank, and status	SQL views generate summary and subject-level reports dynamically
Authentication Mapping	Connect users to academic identities	<code>profiles</code> links authenticated users to students, teachers, or administrators
Security	Restrict records by role and scope	RLS policies and helper functions enforce row-level visibility and modification rules

2.2 Non-Functional Requirements

- The database must preserve integrity through keys and constraints.
- The design must minimize redundancy.
- The schema must support future growth in students, subjects, and classes.
- Sensitive academic data must not be visible to unauthorized users.
- Derived report values should be computed rather than redundantly stored.
- Queries should remain understandable, maintainable, and extensible.

2.3 Users and Data Responsibilities

Role	Data Responsibility
Administrator	Maintains master data and creates academic accounts
Teacher	Inserts or updates marks only within authorized teaching scope
Student	Reads only personal report data

2.4 Core Business Rules

- Each student has a unique `student_id`.
- Each teacher has a unique `teacher_id`.
- Each subject has a unique `subject_name`.
- Each class has a unique `class_name`.
- A mark must be between 0 and 100.
- A subject result is **Pass** if mark ≥ 50 , otherwise **Fail**.
- One student can have only one stored mark per subject.
- A teacher may record marks only within authorized teaching scope.
- Report totals, averages, and rank are derived values and must not be stored in base tables.

3. Database-Oriented System Design

The project was designed around the database as the central component. The interface and application logic are secondary layers used only to submit requests and display results. The academic rules themselves are centered in the database schema and SQL logic.

3.1 Design Philosophy

The design follows four principles:

- **Store base facts only once**
- **Model real relationships explicitly**
- **Compute derived values instead of duplicating them**

- **Push security and validation as close to the data as possible**

3.2 Data-Centered Architecture

Layer	Role
Client Interface	Collects input and displays query results
Application Gateway	Passes validated requests to the database
Database Core	Stores base entities, enforces constraints, executes SQL views, functions, triggers, and RLS policies

In this architecture, the database core is the primary academic asset. The interface is replaceable, but the schema, SQL logic, and access policies define the correctness of the system.

3.3 Current System Architecture

The implemented system follows a three-layer academic data architecture. At the top, the user interface collects academic input and displays reports. In the middle, the application layer forwards validated requests to Supabase/PostgreSQL and returns the resulting output. At the core, PostgreSQL stores the base relations, executes SQL functions and views, applies triggers and indexes, and enforces Row-Level Security policies. Core entities such as students, teachers, subjects, classes, marks, and profiles are stored in normalized relations, while reporting outputs are generated from database views and controlled query execution.

3.4 Current System Flow

The operational flow of the implemented system is database-driven and can be understood through three main transactions. First, an administrator creates academic records such as classes, subjects, teachers, or students, and the database validates foreign keys, uniqueness rules, and structural constraints before commit. Second, a teacher enters or updates marks, after which authorization is checked, the marks relation is updated, and dependent reporting views immediately reflect the new values. Third, an authorized user requests a report, and the database enforces RLS, calculates totals, averages, ranking positions, and pass or fail status, then returns only the permitted result set.

4. Entity Relationship Diagram

The diagram below models the persistent PostgreSQL tables with explicit min-max cardinality labels on each connector and also annotates the reporting views student_subject_report and student_report_summary as derived objects. The dashed view boxes are included for reporting completeness, but they are not stored entities and do not change the formal table cardinalities.

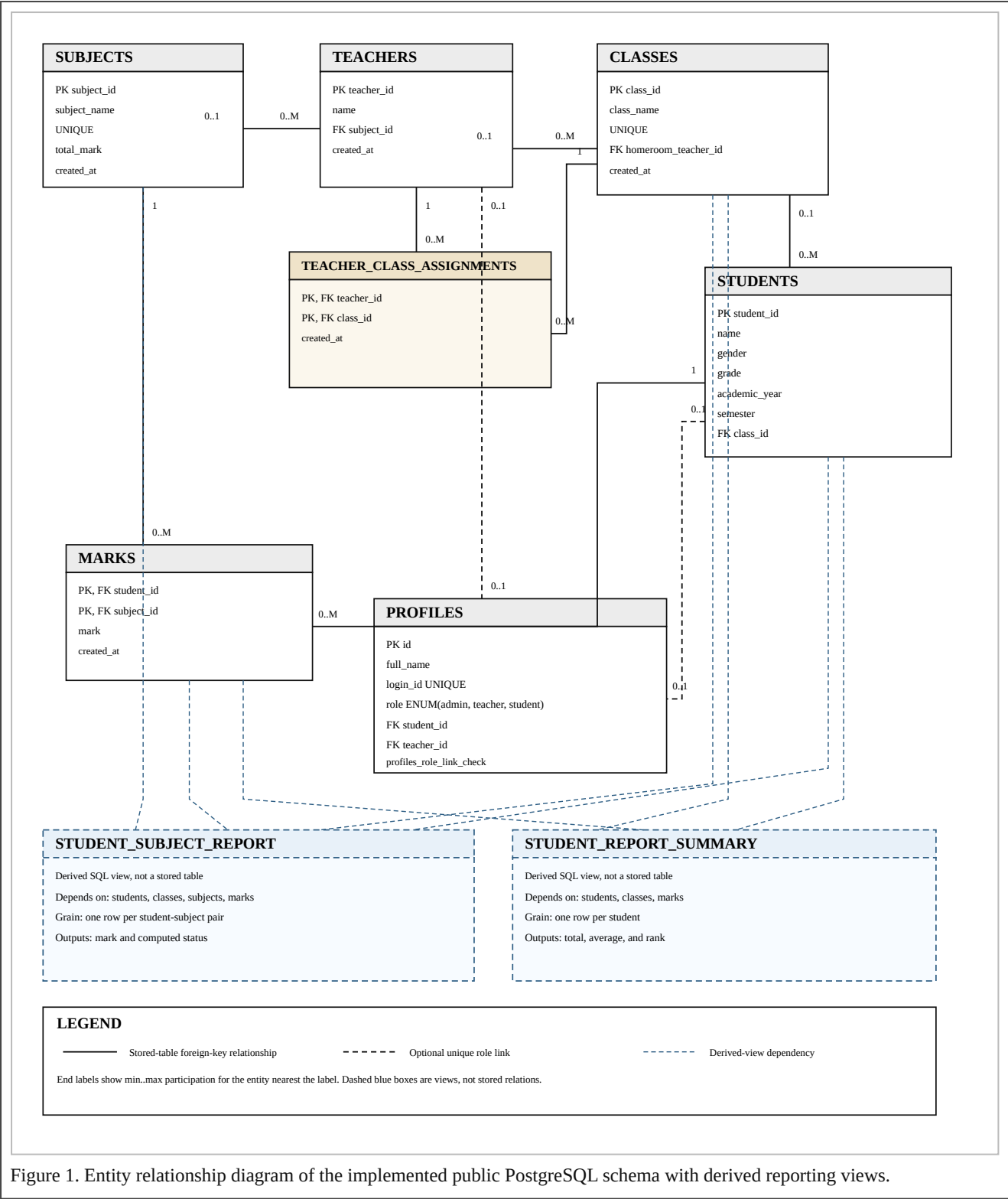


Figure 1. Entity relationship diagram of the implemented public PostgreSQL schema with derived reporting views.

4.1 Core Entities

Entity	Primary Key	Important Attributes
subjects	subject_id	subject_name, total_mark, created_at
teachers	teacher_id	name, subject_id, created_at
classes	class_id	class_name, homeroom_teacher_id, created_at
teacher_class_assignments	(teacher_id, class_id)	created_at

Entity	Primary Key	Important Attributes
students	student_id	name, gender, grade, academic_year, semester, class_id
marks	(student_id, subject_id)	mark, created_at
profiles	id	full_name, login_id, role, student_id, teacher_id

4.2 Relationship Specification

The implemented schema has the following cardinalities and participation rules:

- **teachers.subject_id -> subjects.subject_id**: each teacher references 0..1 subject, while each subject can be linked to 0..M teachers.
- **classes.homeroom_teacher_id -> teachers.teacher_id**: each class references 0..1 homeroom teacher, while each teacher can be linked to 0..M classes in that homeroom role.
- **teacher_class_assignments** resolves the teacher-class assignment relationship: each assignment row references exactly 1 teacher and 1 class, while each teacher and each class can participate in 0..M assignment rows.
- **students.class_id -> classes.class_id**: each student references 0..1 class, while each class can contain 0..M students.
- **marks** resolves the student-subject result relationship: each mark row references exactly 1 student and 1 subject, while each student and each subject can participate in 0..M mark rows.
- **profiles.student_id -> students.student_id**: a student can be linked to 0..1 profile and a profile can point to 0..1 student.
- **profiles.teacher_id -> teachers.teacher_id**: a teacher can be linked to 0..1 profile and a profile can point to 0..1 teacher.

4.3 Derived Reporting Views

- **student_subject_report**: derived from students, classes, subjects, and marks; provides one row per student-subject pair with computed status.
- **student_report_summary**: derived from students, classes, and marks; provides total, average, and rank per student.

4.4 ER Diagram Interpretation

The ERD shows a correct relational separation between master entities, associative and transactional entities, identity data, and derived reporting views. Students, teachers, classes, and subjects are master data; teacher_class_assignments and marks are association and transaction tables; profiles stores identity and authorization metadata; and the two reporting views remain computed outputs rather than stored facts. This separation is important because it avoids storing mixed responsibilities in a single table while still making the reporting layer explicit.

5. Database Design and Normalization

The database was designed to minimize redundancy, preserve integrity, and support dynamic academic reporting. The schema combines natural business identifiers and surrogate keys where appropriate.

5.1 Relational Schema and Key Design

Relation	Key Structure	Database Rationale
subjects	UUID primary key	stable internal identifier for curriculum entities
classes	UUID primary key	avoids dependence on mutable class names

Relation	Key Structure	Database Rationale
teachers	text primary key	institutional teacher ID is already meaningful business data
students	text primary key	institutional student ID is a business identifier
marks	composite primary key	prevents duplicate marks for the same student-subject pair
profiles	UUID primary key	links authenticated identity to academic identity

5.2 Primary Keys, Foreign Keys, and Referential Integrity

Primary Keys

- `subjects(subject_id)`
- `teachers(teacher_id)`
- `classes(class_id)`
- `students(student_id)`
- `profiles(id)`
- `marks(student_id, subject_id)`

Foreign Keys

- `teachers.subject_id -> subjects.subject_id`
- `classes.homeroom_teacher_id -> teachers.teacher_id`
- `students.class_id -> classes.class_id`
- `marks.student_id -> students.student_id`
- `marks.subject_id -> subjects.subject_id`
- `profiles.student_id -> students.student_id`
- `profiles.teacher_id -> teachers.teacher_id`
- `profiles.id -> auth.users.id`

Referential Actions

- `ON DELETE CASCADE` is used where child rows should not survive independently, such as marks belonging to a deleted student.
- `ON DELETE SET NULL` is used where the row remains meaningful even after the parent disappears, such as a class whose homeroom teacher has been removed.

This is a sound design choice because referential actions are selected according to business meaning rather than convenience.

5.3 Constraints and Integrity Rules

The schema includes the following important constraints:

- `subject_name` is unique.
- `class_name` is unique.
- `gender` is restricted to male, female, or other.
- `total_mark > 0` is enforced for subjects.
- `mark` between 0 and 100 is enforced in marks.
- `login_id` is unique in a case-insensitive way through an index on `lower(login_id)`.
- `profiles_role_link_check` guarantees valid role-to-record mapping. A student profile must reference a student and not a teacher, a teacher profile must reference a teacher and not a student, and an administrator profile must reference neither.

These constraints are academically important because they ensure that invalid states are rejected at the database level.

5.4 Normalization Analysis

First Normal Form (1NF)

The schema satisfies 1NF because:

- all attributes are atomic,
- there are no repeating groups,
- subject performance is not stored as multiple columns in `students`,
- relationships are modeled through explicit foreign keys.

Second Normal Form (2NF)

The schema satisfies 2NF because non-key attributes depend on the whole key:

- in `marks`, the attribute `mark` depends on the full composite key (`student_id`, `subject_id`).

Third Normal Form (3NF)

The schema satisfies 3NF because transitive dependencies are removed:

- student data is stored in `students`,
- teacher data is stored in `teachers`,
- subject data is stored in `subjects`,
- class data is stored in `classes`,
- totals, averages, and rank are not stored in base relations.

This eliminates update anomalies and keeps the schema logically clean.

5.5 SQL Views, Functions, and Ranking Logic

The project demonstrates advanced database understanding through SQL objects beyond ordinary tables.

A. `student_subject_report` View

Purpose:

- joins `students`, `classes`, `subjects`, and `marks`,
- shows one row per student-subject pair,
- uses `COALESCE` to treat missing marks as 0,
- uses `CASE` to derive subject-level `Pass` or `Fail`.

Database significance:

- centralizes subject-level reporting logic,
- prevents repeated pass/fail computation in multiple places,
- provides a reusable base for detailed reports and class matrices.

B. `student_report_summary` View

Purpose:

- groups marks by student,
- calculates total using `SUM(mark)`,
- calculates average using `AVG(mark)`,
- calculates rank using `RANK() OVER (ORDER BY SUM(mark) DESC)`.

Database significance:

- separates transactional storage from reporting storage,
- demonstrates aggregate functions and window functions,
- guarantees that report summaries always reflect current marks.

C. Stored Functions

Function	Database Role
<code>current_role()</code>	returns the authenticated user's role
<code>current_teacher_subject_id()</code>	returns the teacher's assigned subject

Function	Database Role
<code>resolve_login_email()</code>	resolves visible login data to internal auth identity
<code>create_student_with_account()</code>	creates student, auth, and profile data consistently
<code>create_teacher_with_account()</code>	creates teacher, assignment, auth, and profile data consistently
<code>change_own_password()</code>	performs protected password update
<code>build_hidden_login_email()</code>	builds internal email format for authentication compatibility

These functions are important because they move critical business logic into the database instead of leaving it entirely to application code.

D. Trigger

The trigger `set_profiles_updated_at` updates `profiles.updated_at` automatically before each update. This is a simple but correct example of using triggers to maintain metadata consistency.

E. Ranking Logic Interpretation

The main ranking logic is implemented in SQL through the `student_report_summary` view. This is academically stronger than storing rank as a column because rank depends on the current dataset and should be recomputed whenever marks change. If future requirements demand ranking by class, grade, semester, or academic year, the same logic can be extended through partitioned window functions.

5.6 Performance Considerations and Indexing

Performance is important even in a university project because poor schema choices create unnecessary long-term cost.

Existing Performance Strengths

- primary keys automatically create indexes,
- unique constraints on subject and class names create lookup indexes,
- a case-insensitive unique index exists on `profiles(login_id)`,
- composite primary keys prevent duplicates efficiently,
- report summaries are derived from source data rather than stored redundantly.

Recommended Additional Indexes

```
create index if not exists idx_students_class_id
on public.students (class_id);

create index if not exists idx_teachers_subject_id
on public.teachers (subject_id);

create index if not exists idx_classes_homeroom_teacher_id
on public.classes (homeroom_teacher_id);

create index if not exists idx_marks_subject_id
on public.marks (subject_id);

create index if not exists idx_marks_student_id_subject_id
on public.marks (student_id, subject_id);
```

Optimization Notes

- filtering should occur as close to the database as possible,
- summary values should continue to be computed from views,
- very large deployments may benefit from materialized views for reporting,
- ranking can be optimized further by partitioning over academic cohorts.

5.7 Why Derived Values Are Not Stored

The schema intentionally does not store `total`, `average`, `rank`, or `status` in transaction tables. This is good database practice because:

- stored derived data becomes inconsistent after updates,
- update operations become more complex,
- duplicated data increases the risk of anomalies,
- SQL views already provide a correct mechanism for deriving such values.

6. SQL Implementation and Database Operations

This section focuses on how the database objects work together to support academic operations.

6.1 Main SQL Objects

Object Type	Name	Purpose
Table	students	stores learner data
Table	teachers	stores teacher data
Table	subjects	stores subject data
Table	classes	stores class data
Table	marks	stores academic transactions
Table	profiles	stores role-linked identity metadata
View	student_subject_report	subject-level reporting
View	student_report_summary	total, average, and rank
Function	create_student_with_account()	protected student creation workflow
Function	create_teacher_with_account()	protected teacher creation workflow
Function	change_own_password()	secure password change
Function	resolve_login_email()	authentication mapping
Trigger	set_profiles_updated_at	metadata maintenance

6.2 Table Creation Example

```
create table if not exists public.marks (
  student_id text not null references public.students(student_id) on delete cascade,
  subject_id uuid not null references public.subjects(subject_id) on delete cascade,
  mark numeric(5,2) not null check (mark between 0 and 100),
  created_at timestamptz not null default timezone('utc', now()),
  primary key (student_id, subject_id)
);
```

This design shows three important ideas:

- referential integrity through foreign keys,
- domain validation through a check constraint,
- uniqueness through a composite primary key.

6.3 Report Computation Example

```
create or replace view public.student_report_summary as
select
  s.student_id,
```

```
s.name as student_name,  
c.class_name,  
coalesce(sum(m.mark), 0) as total,  
coalesce(avg(m.mark), 0) as average,  
rank() over (order by coalesce(sum(m.mark), 0) desc) as rank  
from public.students s  
left join public.classes c on c.class_id = s.class_id  
left join public.marks m on m.student_id = s.student_id  
group by s.student_id, s.name, c.class_name;
```

This view is important because it demonstrates:

- grouping,
- aggregation,
- null handling with COALESCE,
- ranking with a window function.

6.4 Transaction-Oriented Database Workflows

A. Student Creation

`create_student_with_account()` performs one protected workflow that creates:

- a `students` row,
- an `auth.users` row,
- an `auth.identities` row,
- a `profiles` row.

From a database perspective, this is valuable because related data is created in a controlled and consistent way.

B. Teacher Creation

`create_teacher_with_account()` performs a similar multi-table workflow and creates the teacher plus linked authentication/profile records in one controlled operation. This demonstrates coordinated insertion across related relations.

C. Mark Upsert

Mark storage is based on a composite key. When the same student-subject pair is submitted again, the row is updated instead of duplicated. This reflects correct transaction design for academic marks.

6.5 Minimal Role of the Interface Layer

The interface layer is intentionally thin. Its job is only to:

- collect input,
- submit requests,
- display query results.

The important point for this project is that the database, not the interface, owns:

- data structure,
- integrity rules,
- report logic,
- access control,
- identity mapping,
- error enforcement.

For that reason, the academic merit of the project remains database-centered.

7. Security, RLS, and Exception Handling

Security is one of the strongest database aspects of the project because it is enforced close to the data rather than only in the interface.

7.1 Security Model

The system uses:

- role-aware identity in `profiles`,
- helper functions to determine user scope,
- Row-Level Security policies on key relations,
- protected stored functions for sensitive operations.

7.2 Row-Level Security (RLS) Explanation

RLS is enabled on:

- `teachers`
- `classes`
- `subjects`
- `students`
- `marks`
- `profiles`

Why RLS Is Important

RLS makes the database itself responsible for determining which rows a user may read or modify. This is especially important in academic systems because marks and profiles are sensitive.

Helper Functions Used by RLS

- `current_role()`
- `current_teacher_subject_id()`

These functions make the policies clearer, reusable, and easier to maintain.

7.3 Policy Interpretation

- administrators can broadly manage academic master data,
- teachers can read only permitted academic rows,
- teachers can insert or update marks only within their authorized teaching scope,
- students can read only their own records.

This means the database remains secure even if a user tries to bypass the normal interface flow.

7.4 Validation and Exception Handling

Database-level protections include:

- check constraints,
- unique constraints,
- foreign keys,
- RLS policies,
- stored-function exceptions.

Typical rejected cases include:

- duplicate login IDs,
- duplicate academic IDs,
- invalid mark values,
- unauthorized account creation,
- wrong current password,
- mark entry outside teacher scope.

These behaviors demonstrate that the database does not trust external input blindly.

8. Testing and Validation

Testing was evaluated from the perspective of database correctness, not only interface behavior.

8.1 Database-Focused Testing Strategy

The testing strategy should cover:

- key and constraint validation,
- referential integrity,
- view correctness,
- function correctness,
- RLS enforcement,
- boundary values,
- duplicate-prevention behavior.

8.2 Core Integrity Test Cases

Test Scenario	Expected Result
Insert duplicate subject name	rejected by uniqueness constraint
Insert duplicate class name	rejected by uniqueness constraint
Insert mark above 100	rejected by check constraint
Insert mark below 0	rejected by check constraint
Insert duplicate student-subject mark pair	existing row is updated rather than duplicated
Delete student with stored marks	dependent marks are removed by cascade
Delete homeroom teacher	class row remains, homeroom field becomes NULL
Query summary view after mark insertion	total, average, and rank reflect the new data

8.3 Advanced Security and Edge Cases

Test Case	Purpose	Expected Result
Teacher inserts mark for unassigned subject	authorization check	rejected
Teacher inserts mark for unauthorized student record	authorization check	rejected
Student attempts to read another student's record	ownership check	blocked by RLS
Create student with login ID differing only by letter case	case-insensitive uniqueness test	rejected
Change password with incorrect current password	protected function validation	rejected
Change password to same password	security rule validation	rejected
Query reports when marks are missing	null-handling validation	values default correctly through COALESCE
Insert teacher with duplicate login ID in protected workflow	identity integrity validation	rejected by unique index

8.4 Validation Summary

The database satisfies the main objectives of an Advanced Database project because it demonstrates:

- correct relational modeling,
- controlled transaction storage,
- dynamic reporting through SQL,
- authorization through RLS,
- consistent handling of invalid input.

8.5 Testing Gap

The current project does not yet include a full automated database test suite. A stronger future extension would add:

- SQL unit tests for functions,
- policy tests for RLS,
- regression tests for views,
- transactional tests for insert and update workflows.

9. Database-Relevant Screenshots and Evidence

This section keeps only screenshots that directly support database understanding. The purpose is not to show interface beauty, but to show evidence of database-backed operations and derived reporting.

Figure 2. Student Data Entry Backed by Multi-Table Creation

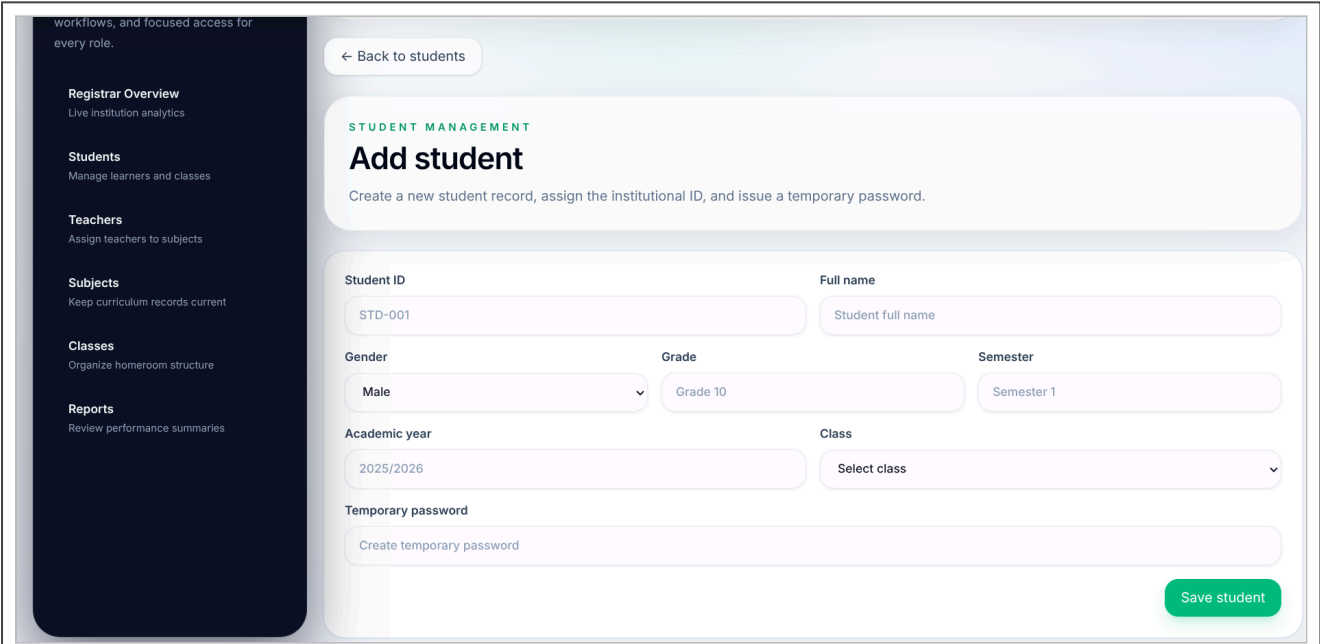


Figure 2. Student data entry backed by multi-table creation.

Database meaning: This page represents insertion into the academic schema. The entered values map to students, while the account creation process also creates linked authentication and profiles records through `create_student_with_account()`.

What it proves: The system does not store student identity in only one isolated table; it coordinates academic and authentication data in a controlled database workflow.

Figure 3. Mark Entry Using Composite-Key Storage

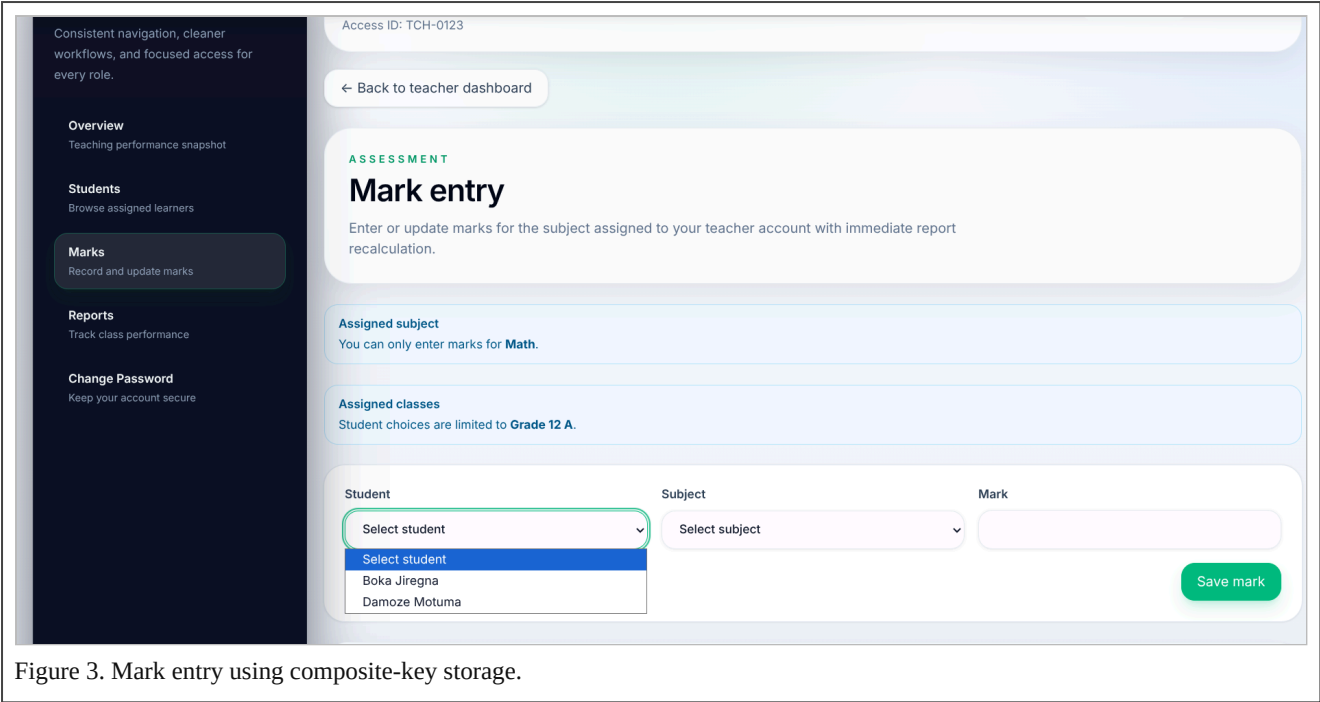


Figure 3. Mark entry using composite-key storage.

Database meaning: This workflow inserts or updates marks(student_id, subject_id, mark). Because the table uses a composite primary key, the same student-subject pair cannot create duplicate rows.

What it proves: The design treats marks as transactions and enforces uniqueness and authorization at the database level.

Figure 4. Class Performance Matrix Generated from SQL Views

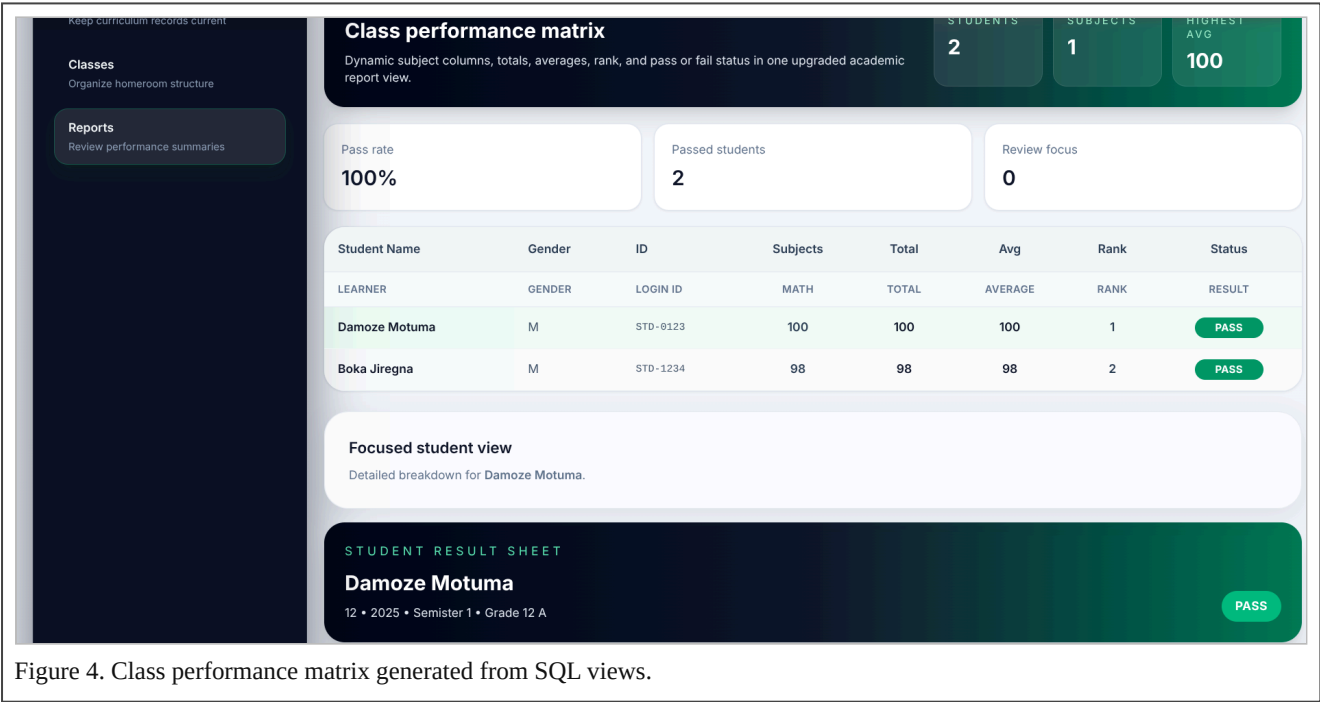
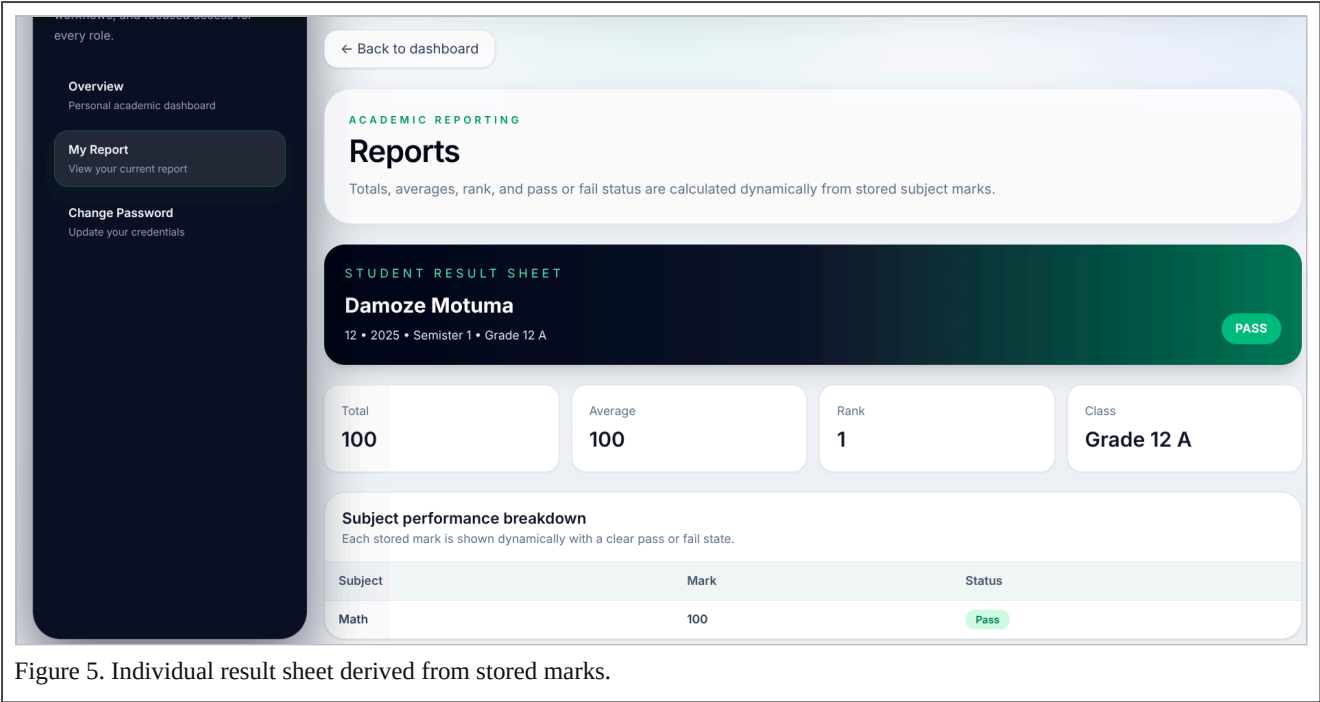


Figure 4. Class performance matrix generated from SQL views.

Database meaning: The displayed totals, averages, rank, and status are not stored directly in master tables. They are derived from student_subject_report and student_report_summary.

What it proves: The screenshot is evidence that reporting is view-based and dynamically computed from normalized data.

Figure 5. Individual Result Sheet Derived from Stored Marks



Database meaning: The detailed result sheet is assembled from summary-level and subject-level SQL outputs rather than from manually maintained report columns.

What it proves: The database can support both aggregate and detailed reporting without duplicating academic data.

9.1 Screenshot Section Interpretation

Taken together, the screenshots demonstrate:

- correct insertion into related tables,
- transactional mark storage,
- dynamic SQL-based reporting,
- separation between base tables and derived report output.

10. Unique and Innovative Database Features

The project contains several database features that make it stronger than a simple CRUD design.

10.1 Separation of Base Tables and Derived Views

The system stores base facts in tables and derives report facts in views. This is a strong relational design decision because it reduces redundancy and inconsistency.

10.2 Homeroom-Teacher Referential Modeling

The use of `classes.homeroom_teacher_id` models teacher-to-class advisory responsibility through a direct foreign-key link and demonstrates clean referential design.

10.3 Case-Insensitive Login ID Uniqueness

The unique index on `lower(login_id)` prevents duplicates that differ only by letter case. This is a useful real-world integrity enhancement.

10.4 Role-Link Constraint in profiles

The `profiles_role_link_check` constraint ensures that role-to-record mapping remains valid. This is more rigorous than relying on application code alone.

10.5 RLS Helper Functions

The use of helper functions inside RLS policies is an advanced design choice because it keeps policies expressive and maintainable.

10.6 Protected Multi-Table Account Creation

The account-creation functions show how complex academic workflows can be encapsulated in stored logic instead of spreading the same rules across multiple interface actions.

11. Conclusion

This project demonstrates a strong understanding of database design for academic record management. Its main achievement is not the existence of a web interface, but the creation of a coherent relational schema supported by keys, constraints, views, functions, triggers, indexes, and Row-Level Security. The project shows how normalized storage and dynamic SQL reporting can solve a real institutional problem while preserving integrity and security.

From an Advanced Database perspective, the most important strengths of the work are:

- correct modeling of academic entities and relationships,
- proper handling of transactional marks,
- dynamic report generation through SQL,
- controlled access through RLS,
- avoidance of redundant stored summaries.

The project therefore stands as a database-centered academic solution rather than a frontend-centered system demonstration. A lightweight interface may change over time, but the database logic, schema quality, and SQL design remain the real foundation of the system.

12. Appendix

Appendix A. Main Database Objects

Object Type	Name	Importance
Table	students	stores learner master data
Table	teachers	stores teacher master data
Table	subjects	stores subject master data
Table	classes	stores class master data
Table	marks	stores academic transaction data
Table	profiles	stores role-linked identity metadata
View	student_subject_report	provides subject-level reporting
View	student_report_summary	provides summary-level reporting
Function	create_student_with_account()	creates linked student and profile records
Function	create_teacher_with_account()	creates linked teacher and assignment records
Function	resolve_login_email()	supports identity resolution
Function	change_own_password()	secures password changes
Function	current_role()	supports RLS
Trigger	set_profiles_updated_at	maintains update timestamps

Appendix A.1 Key Indexes and Constraint Summary

Database Object	Type	Academic Importance
students_pkey	primary key	guarantees that each student record is uniquely identifiable

Database Object	Type	Academic Importance
teachers_pkey	primary key	guarantees that each teacher record is uniquely identifiable
marks_pkey (student_id, subject_id)	composite primary key	prevents duplicate marks for the same student in the same subject
subjects_subject_name_key	unique constraint	avoids duplicate subject definitions
classes_class_name_key	unique constraint	preserves one canonical record per class
profiles_login_id_lower_idx	case-insensitive unique index	prevents logically duplicated login identities
marks_mark_check	check constraint	ensures entered marks remain within the valid academic range
profiles_role_link_check	check constraint	ensures profile roles match the linked academic entity
foreign key constraints	referential integrity	stop orphan records and preserve relational correctness

Appendix A.2 Centrally Enforced Database Rules

- mark values must remain within the permitted numeric range,
- a student-subject mark pair can exist only once,
- report totals, averages, and ranks are derived rather than stored,
- role-based access is enforced in the database through RLS policies,
- sensitive academic data is exposed only through authorized queries and functions.

Appendix A.3 Role-to-Database Access Summary

Role	Main Database Scope	Access Characteristics
Administrator	students, teachers, classes, subjects, profiles	creates and maintains master data through protected workflows and authorized inserts
Teacher	marks, reporting views, permitted class data	can read only permitted academic rows and update marks within authorized teaching scope
Student	personal reporting views and linked profile data	can read only personal result information under row-level restrictions

This role-to-database mapping is important because it shows that access control is not merely a user-interface feature. Each role operates against a different subset of the schema, and those boundaries are enforced in the database by RLS and helper functions rather than by client-side assumptions alone.

Appendix A.4 Derived Reporting Attributes

Derived Attribute	Source Logic	Reason It Is Not Stored Directly
total	sum(mark) across the student result set	avoids inconsistent totals after mark updates
average	avg(mark) across recorded subjects	prevents duplication of recalculable values
rank	window function over ordered totals	keeps ranking synchronized with current transactions
status	conditional SQL expression from the mark value	avoids storing pass/fail flags that can be recomputed

These derived attributes demonstrate a core advanced-database principle used throughout the project: transactional facts are stored in base tables, while reporting facts are generated by SQL at query time. This keeps the schema normalized and reduces update anomalies.